

EngiSync — UX, Product & Architecture Review

Findings, challenges, and a redesign proposal — before any code is written

Prepared for Tafadzwa Musendo

27 July 2026

0. How to read this document

Part A — Findings

- Finding 1 (Critical): A “project” is three different things in your app
- Finding 2 (Critical): Every feature has two homes and neither is authoritative
- Finding 3 (High): The dashboard is a marketing page
- Finding 4 (High): There is no project lifecycle in the schema
- Finding 5 (High): The role model cannot express what you’re asking for
- Finding 6 (Medium): There is no activity infrastructure
- Finding 7 (Medium): Duplicate resource surfaces
- Finding 8 (Medium): Scalability constraints already present

Part B — Six challenges to your brief

- Challenge 1: “AI should exist everywhere” is currently impossible, and would be expensive
- Challenge 2: The Engineering Score is the riskiest idea in your brief
- Challenge 3: AI cannot do plagiarism detection — remove it
- Challenge 4: Be careful what you let AI say about engineering calculations
- Challenge 5: Six roles is over-engineering — ship three
- Challenge 6: You have three competing scoring systems

Part C — The redesign

- C.1 The core model: one noun
- C.2 Navigation: commit to the container axis
- C.3 The dashboard: answer one question
- C.4 Project lifecycle
- C.5 Onboarding
- C.6 Leader workspace
- C.7 Backend architecture

Part D — Roadmap

Part E — What I need from you before writing code

0. How to read this document

You asked me to challenge your assumptions rather than implement your list. So this document is deliberately split:

- **Part A — Findings.** What is actually wrong, evidenced from the code, not from impressions.
- **Part B — Challenges.** Six things in your brief I think are wrong or dangerous, with alternatives.
- **Part C — The redesign.** The proposed target architecture.
- **Part D — Roadmap.** A resequenced plan, and why your phase order won't work as written.

Nothing has been implemented. The single most important thing in here is Finding 1 — if you read nothing else, read that.

Part A — Findings

Finding 1 (Critical): A “project” is three different things in your app

This is the actual root cause of “users don't understand how to use the application,” and it is not a UI problem.

In the database there is exactly **one** entity: `Workspace`. But the application presents it under three different names, in two different page trees:

What the user sees	Route	What it shows
“Groups”	<code>/dashboard/workspaces/[id]</code>	Members, join codes, invites, access settings, collaborations
“Projects”	<code>/dashboard/projects/[id]</code>	Objectives, scope, milestones, risks, deliverables

I verified this directly: `listProjects()` in `src/lib/project.ts` calls `listWorkspacesForUser()` and returns the workspace `id` unchanged. `/dashboard/workspaces/abc` and `/dashboard/projects/abc` are the same record. The user is looking at two halves of one thing, through two doors, with nothing telling them the doors lead to the same room.

The consequences cascade:

- A group leader who wants to add a milestone goes to “Groups,” doesn't find it, and concludes the feature doesn't exist.
- The sidebar spends two of its top-level slots on one concept.
- Everything downstream — dashboard, onboarding, leader tools, analytics — has to answer “which one do I link to?” and there is no correct answer.

Your instinct that the problem is “too many buttons” was close but one level too shallow. Users aren't overwhelmed by button *count*; they're unable to build a mental model because the app's central noun is incoherent. My previous navigation pass (7 sections → 4, plus a command

palette) helped users *find* pages faster — but finding pages faster is not the same as understanding the product. That work was treating the symptom.

This must be fixed first. Onboarding, the intelligent dashboard, and the leader workspace all depend on there being a single, well-defined thing called a project.

Finding 2 (Critical): Every feature has two homes and neither is authoritative

Of 16 navigation destinations, **eight** are cross-workspace aggregate views of features that *also* exist inside a workspace: Tasks, Calendar, Meetings, Budget, Analytics, Files, Collaboration, AI Assistant.

So “my tasks” lives at `/dashboard/tasks` **and** inside `/dashboard/workspaces/[id]`. Same for meetings, budget, files, analytics. The user has to learn, per feature, which view is the “real” one — and the answer differs by feature.

This is an **axis collision**: the app is simultaneously organised by *feature* (top-level nav) and by *container* (inside a workspace), and it never commits to either. Notion, Linear and GitHub all commit: GitHub organises by repository, with a thin cross-repo layer on top (`/issues`, `/pulls`) that is explicitly framed as “yours, across everything.” Your app has the two layers but doesn’t visually or conceptually distinguish them.

Finding 3 (High): The dashboard is a marketing page

`src/app/dashboard/page.tsx` renders an `ImageSlider` (rotating hero photographs) and `RotatingText` above the actual content. This is a **logged-in** dashboard.

A hero carousel is a device for persuading anonymous visitors. The user is already persuaded — they logged in. Every pixel of that carousel is occupying the most valuable real estate in the product to answer a question the user isn’t asking. Your instinct here is exactly right and I’d go further than you did: the carousel and rotating text should be deleted, not relocated.

Finding 4 (High): There is no project lifecycle in the schema

You want an Idea → Proposal → Planning → Design → Simulation → Implementation → Testing → Documentation → Presentation → Completed timeline.

The `Workspace` model has `objectives` and `scope` but **no stage or status field at all**. `Milestone`, `Deliverable` and `ProjectRisk` exist but are unordered flat lists with no relationship to a lifecycle. There is currently no way to answer “what stage is this project at?” — which also means there is no way to answer “are we behind schedule?”, which is one of the four user goals you named in your brief.

This is a genuine gap, not a UI gap, and it’s the highest-value *new* feature in your entire brief.

Finding 5 (High): The role model cannot express what you're asking for

`WorkspaceRole` has exactly two values: `LEADER` and `MEMBER`. `SystemRole` has `GUEST`, `INDIVIDUAL`, `ADMIN`. The supervisor role is handled by a separate ad-hoc flag threaded through components as `isSupervisor`.

So the permission system is currently: two workspace roles, three system roles, and one boolean bolted on the side. Authorization logic is scattered across server actions rather than centralised. Before adding roles, this needs one consolidated policy layer — otherwise each new role multiplies the number of places a permission check can be forgotten. (See Challenge 5, where I argue you should add *fewer* roles than you asked for.)

Finding 6 (Medium): There is no activity infrastructure

You want a live activity feed in every workspace. There is an `AuditLog` model, but it is security-oriented (actor, action, target, IP) and not designed to be read by end users — and I recently made it *redact* its target field on account deletion, which is correct for audit but wrong for a feed.

A user-facing activity stream is a different concern from a security audit log and needs its own model. Conflating them is a mistake I'd rather not make quietly.

Finding 7 (Medium): Duplicate resource surfaces

`/dashboard/resources` (“Resource Library” — your own uploaded files) and `/dashboard/departments/[id]/resources` (“Resource Hub” — curated department learning material) are two unrelated features whose names are near-synonyms. In my consolidated nav I renamed one to “Files,” which reduced the collision but didn't resolve it. They should be clearly separated as **Files** (things you made) and **Library** (things you learn from).

Finding 8 (Medium): Scalability constraints already present

- **Rate limiting is in-memory**, so it resets on every serverless cold start and isn't shared across instances. On Vercel this means the limit is roughly per-instance, not per-user. Needs Redis before real multi-department load.
- **Analytics are computed on request** by aggregating tasks, attendance and contributions live. This is fine for one group and will not be fine for a lecturer viewing thirty groups. Needs caching or precomputation.
- **Files** are stored via a model that assumes local/blob access; a department-wide rollout needs S3 or Supabase Storage with signed URLs.

Part B — Six challenges to your brief

You asked me to challenge you where appropriate. These are the places where I think following your brief literally would produce a worse product, or a harmful one.

Challenge 1: “AI should exist everywhere” is currently impossible, and would be expensive

You listed roughly 20 embedded AI touchpoints, several of which fire automatically (AI review on every document upload, AI insights on every dashboard load, AI recommendations in the resource hub).

Three problems:

1. **AI is currently disabled** in this app at your instruction, and the API key removed.
2. **The free Gemini tier is ~15 requests/minute.** A single department with thirty active students would exhaust that during one lecture.
3. **We have already been bitten by exactly this.** The Resource Hub previously called the AI on every page load; it was silently burning quota and causing the 429 errors you reported. I fixed it by making it opt-in. Your brief would reintroduce that pattern in twenty places at once.

Counter-proposal — the three rules for AI placement:

- **Never on page load.** AI output is *requested*, never a side effect of navigation.
- **Cache and version everything.** An AI document review is computed once per document version and stored, not recomputed per viewer. A lecturer opening thirty reports should trigger zero new AI calls.
- **Queue anything automatic.** If a document review really should happen on upload, it goes through a background job with a per-user daily budget, not an inline request.

This keeps “AI feels present everywhere” while making the cost bounded and predictable. Without these rules, “AI everywhere” means “the app is broken by lunchtime.”

Challenge 2: The Engineering Score is the riskiest idea in your brief

You propose a single 0–100 score per student, computed from task completion, attendance, communication, documentation, testing, deadlines, participation, project quality and issue resolution — and you note it should “simplify evaluation for lecturers.”

That last clause is the problem. Three objections:

It will be gamed, and the gaming is worse than the metric. “Communication” measured by message count means students post filler to raise their score. “Participation” measured by activity means the student who thinks for two days and then commits the correct solution scores below the one who commits twenty times. You will be training students to perform visible activity rather than good engineering — the exact opposite of what an engineering department wants.

It disadvantages people for reasons unrelated to competence. A student with poor connectivity, a part-time job, or who works in intense focused blocks rather than daily increments will score low while doing excellent work. In a Zimbabwean university context specifically, intermittent connectivity is a real and unevenly-distributed constraint — the score would systematically penalise students for their data situation.

Composite scores launder their assumptions. Once a lecturer sees “Brian: 61/100,” the weighting decisions you made in a helper function have quietly become an academic

judgement, and nobody — including the lecturer — can see that “communication” was weighted at 15%.

Counter-proposal: Keep the underlying signals, kill the composite *for staff*.

- Show the **student** their own dashboard of signals, framed as “things that might need attention,” never as a grade, never a single number.
- Show the **lecturer** the raw signals side by side (tasks completed, last contribution date, attendance) — the same information, without the false precision of a rank.
- If you want a single indicator, put it on the **project**, not the person. “Project health: at risk” is actionable and blames no one. “Brian: 61” is a verdict.

This preserves everything useful about your idea and removes the part that could genuinely harm a student’s academic record.

Challenge 3: AI cannot do plagiarism detection — remove it

Your document-review checklist includes “Possible Similarity.”

A language model has no corpus to compare a document against. Asked whether text is plagiarised, it will produce a confident, fluent, **fabricated** answer. In an academic setting, a false plagiarism flag attached to a student’s report is a serious harm — potentially a disciplinary matter — generated by a system that was guessing.

This must be removed. If plagiarism detection is genuinely required, it needs a real similarity service (Turnitin and equivalents) with an actual index. There is no acceptable LLM-based version of this feature.

Challenge 4: Be careful what you let AI say about engineering calculations

Your checklist also includes “Calculation Consistency” and “Safety Concerns” on uploaded documents.

For a civil engineering student uploading a bridge load calculation, an AI that says “calculations appear consistent” is making a claim it cannot support, in a domain where being wrong has physical consequences, to a reader who is still learning and may not catch the error.

I’m not saying remove it — AI feedback on drafts is genuinely useful. I’m saying constrain it:

- Frame all output as “**draft feedback, not verification**”, prominently, in the UI and not just in a tooltip.
- Have it **raise questions rather than issue verdicts**: “Section 4 doesn’t state the assumed load factor — is that intentional?” is helpful. “Calculations are correct” is not something it may say.
- **Never** let AI output mark a deliverable as approved, or feed into anything a supervisor sees as a pass/fail signal.

Challenge 5: Six roles is over-engineering — ship three

You proposed: Department Administrator → Lecturer/Supervisor → Group Leader → Assistant Leader → Member → Guest.

“Assistant Leader” is a whole permission tier invented to express one thing: *a second person who can approve stuff when the leader is unavailable*. That’s a capability, not an identity. Adding it as a role means every future permission check has to reason about six tiers, and the odds of getting all of them right in every server action drop accordingly (see Finding 5 — the checks are currently scattered).

Counter-proposal: three workspace roles, plus explicit delegation.

- LEADER, MEMBER, VIEWER (viewer covers supervisors observing, and guests on shared links).
- Any member can be granted specific **capabilities** by the leader: `can_approve`, `can_manage_budget`, `can_invite`. Your “Assistant Leader” is then just a member with `can_approve` — visible in the UI as a badge, without a new tier in the authorization logic.
- Department Administrator and Supervisor stay at the **system/department level**, where they already belong, rather than being squeezed into the workspace hierarchy.

Fewer tiers, same expressiveness, dramatically fewer places to get authorization wrong.

Challenge 6: You have three competing scoring systems

Your brief separately asks for: an Engineering Score (0–100 per student), Project Analytics (health, risk, productivity per member), and Gamification badges (Top Contributor, Most Helpful Member).

All three rank students against each other using overlapping inputs. Together they will feel less like a workplace tool and more like a leaderboard, which cuts directly against your stated goal of “professional, engineering-inspired, not another Trello.”

Counter-proposal: one scoring concept, applied to projects, not people — **Project Health**. Keep badges, but make them *non-comparative* and achievement-based (“Documented every milestone”), never ranked (“Top Contributor” implies a bottom contributor). Drop the per-student composite entirely, per Challenge 2.

Part C — The redesign

C.1 The core model: one noun

A Project is the unit of everything. A project has members (that’s what “group” meant), a lifecycle stage, tasks, meetings, documents, a budget, and analytics. “Workspace,” “Group” and “Project” collapse into a single concept with a single URL space:

<code>/projects/[id]</code>	overview – health, stage, activity, next actions
<code>/projects/[id]/tasks</code>	
<code>/projects/[id]/docs</code>	
<code>/projects/[id]/meetings</code>	
<code>/projects/[id]/budget</code>	
<code>/projects/[id]/team</code>	members, invites, join codes, roles
<code>/projects/[id]/insights</code>	analytics, risks, AI evaluation

`/dashboard/workspaces/*` and `/dashboard/projects/*` both redirect here permanently. This is one migration and it removes an entire category of confusion.

C.2 Navigation: commit to the container axis

Resolve the axis collision (Finding 2) by making the app **project-first**, with an explicitly-labelled personal layer on top — the GitHub model.

Desktop sidebar — five items, permanently:

Home	"what should I do right now"
My Work	MY tasks/meetings/deadlines across all projects
Projects	list → the project workspace above
Library	department resources + learning material
Department	announcements, people, admin (if applicable)

Everything else — budget, analytics, documents, discussions, quizzes, AI evaluation — lives **inside a project**, because that is where it belongs and where users will look for it once the noun is coherent. Settings moves to the avatar menu. Admin and Supervisor appear as additional items only for those roles.

The distinction that makes this work: **“My Work” is a lens, “Projects” is a place.** My Work is explicitly framed as “yours, pulled from everywhere” — which is why it doesn’t feel like a duplicate of the project’s task list.

Mobile: bottom bar of Home · My Work · Projects · More . Four items, not five — the fifth is always the overflow.

The ⌘K command palette stays and becomes more important, since it’s now the escape hatch for power users who want to jump straight to a project’s budget.

C.3 The dashboard: answer one question

Delete the carousel. Replace the whole page with a single decision-support surface:

Good morning, Tafadzwa

TODAY

Finish Bridge Design Report due 17:00
Team standup in 2 hrs
+ 3 more tasks due today
[Continue where you left off →]

YOUR PROJECTS

Bridge Load Analysis
Design stage · Healthy · 68% complete
● 2 tasks need you

Concrete Lab Report
Testing stage · At risk · behind by 4d
● Awaiting your approval

RECENT ACTIVITY

John uploaded BeamAnalysis.pdf 2h
Sarah completed "Load calculations" 4h

Three blocks, in priority order: what’s urgent, what’s mine, what changed. Note there are no charts on the dashboard — charts answer “how are we doing overall,” which is a different question, and it belongs in project insights.

C.4 Project lifecycle

Add a `stage` field to the project with your ten stages, plus `stageEnteredAt` so time-in-stage becomes measurable (this is what makes “are we behind?” answerable). Render as a horizontal stepper on the project overview.

Two design notes:

- **Ten stages is a lot for a one-semester project.** I’d ship with stages collapsible into four phases (Define / Design / Build / Deliver) so the stepper stays readable on mobile, with the ten sub-stages visible on the detail view.
- **Stages should be advisory, not gates.** Blocking a student from uploading a document because the project is in “Planning” would be infuriating and pedagogically wrong. Show the stage; don’t enforce it.

C.5 Onboarding

Your interactive checklist design is good and I’d build it close to as specified. Two changes:

- **Make it resumable and non-blocking.** Students will start on a phone, get interrupted, and come back on a laptop. It must never trap them in a modal — the existing “Get set up” checklist is already state-aware and dismissible; extend that rather than replacing it with a wizard.
- **Branch on the first answer.** A student joining an existing group should never see “Create Project” — their path is Join Department → Join Group → See your first task. Three steps, not six. Only leaders get the six-step path.

C.6 Leader workspace

Your list here is strong and I’d largely build it as specified, with one reframing: the leader dashboard should lead with **things that are blocked on the leader** (pending approvals, join requests, unassigned tasks) rather than with statistics. A leader’s actual job in this app is unblocking people. Statistics go below the fold.

C.7 Backend architecture

Recommended before the feature work, because they get harder later:

- **One policy module.** Every authorization decision moves into a single `can(user, action, resource)` layer. This is a prerequisite for the role changes, not a nice-to-have.
- **Separate Activity from AuditLog.** New user-facing model for the feed; audit log stays security-only.
- **Redis for rate limiting** (Upstash works on Vercel’s free tier).
- **Cache the analytics.** Precompute project health nightly plus on-write invalidation. Required before a lecturer can view thirty groups.
- **AI job queue** with per-user budgets, per Challenge 1.
- **Move files to S3/Supabase Storage** with signed URLs before department-wide rollout.

Part D — Roadmap

Your ten phases are reasonable but the order won't work: onboarding (your Phase 3) and the dashboard (Phase 4) both have to link users to "a project," and until Finding 1 is fixed there is no single thing to link to. Building onboarding first means building it twice.

Resequenced, with a verified-working build at every phase boundary:

Phase	Work	Why here
1	Unify Workspace/Project/Group into one entity + URL space. Redirects from old routes.	Everything else depends on the noun being coherent. Nothing new is built.
2	Navigation to five items; project-scoped sub-nav; kill the dashboard carousel.	Cheap, immediately visible, and validates Phase 1 with real use.
3	Policy layer + Activity model.	Unglamorous, but Phases 6–8 all depend on both. Doing it now avoids retrofitting permissions.
4	Project lifecycle stages + interactive stepper.	The highest-value new feature, and a prerequisite for meaningful health/risk analytics.
5	Intelligent dashboard + activity feed.	Now has real data to display: stages, health, activity.
6	Onboarding flow (branching, resumable).	Built once, against the final IA.
7	Leader workspace.	Needs the policy layer and approvals from Phase 3.
8	AI integration, under the three rules from Challenge 1.	Deliberately late: cached, queued, opt-in. Requires the queue from Phase 3.
9	Analytics + project health, with caching.	Needs lifecycle and activity data to be meaningful.
10	Achievements (non-comparative), polish, accessibility, performance.	Last, and smallest — per Challenge 6.

Phases 1–3 produce almost no visible new features. That is intentional and it is the part most likely to be skipped. Skipping them means building the dashboard, onboarding and leader tools on top of an incoherent data model and rebuilding all three later.

Part E — What I need from you before writing code

Six decisions, all of them from Part B:

1. **Engineering Score:** drop the per-student composite and keep project-level health? *(Recommended)*
2. **Plagiarism/“similarity” check:** remove entirely? *(Strongly recommended — false accusation risk)*
3. **Roles:** three roles + capability grants, instead of six tiers? *(Recommended)*
4. **AI:** accept the three rules — never on load, always cached, queued when automatic?
5. **Lifecycle:** ten stages grouped into four phases for display?
6. **Phase 1:** confirm you want the Workspace/Project unification first, accepting that it ships with no new visible features.

Once these are settled I'll start on Phase 1 and hand you a build to verify at each phase boundary.